

Design Note: LLD Practice Platform

CipherSchools Engineering Assignment — Shubham Dubey

1. MVP Overview

The platform is a focused LLD practice tool built around a single loop: a learner picks a problem, writes a text-based design submission, submits it, and receives structured feedback scored against a fixed rubric. Every attempt is stored, so the learner can see previous submissions, compare scores across attempts, and track whether they are improving on specific design dimensions over time.

The scope is deliberately narrow — 3-5 problems, one submission format (text), one evaluation flow (deterministic checks followed by AI rubric scoring), feedback, and attempt history. Nothing more. The goal is to do the practice loop well, not to build an LMS or an assessment platform.

2. User Flow

- Problem selection — The learner sees a list of LLD problems (Parking Lot, Elevator, Vending Machine). Each problem has a title, requirements description, and enough context to attempt it.
- Attempt — The learner starts an attempt. A monospace textarea opens where they write their design: classes, interfaces, responsibilities, relationships, assumptions, and reasoning. No diagram tool, no code compiler — just text.
- Submission — The learner submits. The system runs deterministic checks first (non-empty, minimum length, contains class/interface-like tokens, duplicate detection via content hash). If checks pass, the submission is stored and evaluation begins.
- Evaluation — The AI evaluator scores the submission against a fixed rubric. Each dimension returns a score, evidence quoted from the submission, a concern, a suggestion, and a confidence level. Status transitions through Submitted, Evaluating, Completed (or Failed).
- Feedback review — The learner sees structured feedback: per-dimension scores, evidence quotes from their own submission, and actionable suggestions. No opaque single score.
- History and delta — The learner can view all previous attempts on a problem. If multiple attempts exist, a delta view shows what changed and whether scores per dimension went up or down.

The flow is intentionally linear: *Choose problem* → *Write design* → *Submit* → *Get feedback* → *Review* → *Try again*. Every step either directly helps the learner practise or stores evidence for the improvement loop. Anything that doesn't serve this loop was cut.

3. Domain Model & Key Classes

The core domain has four entities. Each earns its place by owning a distinct responsibility — I challenged each one against the "do I really need this abstraction?" test from the helping guide.

Problem

- Owns: problem identity, title, requirements description, difficulty level, and the rubric template used to evaluate submissions against it.
- Why it exists: The problem is the source of truth for what the learner is designing and which dimensions matter. Different problems can weight rubric dimensions differently — a Parking Lot may emphasise state management, while a Vending Machine emphasises the state machine pattern.

Attempt

- Owns: the relationship between a learner and a problem for a single practice session — when it started, which problem, and a reference to the submitted solution.
- Why it exists: An attempt is not a submission. The learner can start an attempt, draft their design, and not submit yet. The attempt tracks the session; the submission tracks what was actually turned in. This separation means a half-finished draft doesn't trigger evaluation.

Submission

- Owns: the actual content the learner submitted, a content hash for duplicate detection, the submission timestamp, and the evaluation status (Submitted, Evaluating, Completed, Failed).
- Why it exists: Submission is stored before evaluation starts — so if the AI evaluator fails, the submission isn't lost. The content hash enables idempotency: if the same content is submitted twice, we don't re-evaluate. This directly answers the brief's "what should happen if evaluation fails" and "avoid duplicate processing" requirements.

Evaluation

- Owns: the results — a list of RubricResult objects, one per rubric dimension. Each RubricResult has: criterion name, score (1-5), evidence (verbatim quote from the submission), concern (what's weak), suggestion (how to improve), and confidence (how sure the AI is).
- Why it exists: This is the heart of the feedback model. A single opaque "AI score" is exactly what the helping guide says not to build. Breaking evaluation into per-dimension results with evidence gives the learner actionable, specific feedback — they can see that coupling scored low and the evidence shows which classes are tightly coupled.

- Evidence verification: After the AI returns its evaluation, a deterministic check verifies that evidence quotes actually appear in the submission. If a quote doesn't match, `evidenceVerified` is set to false. This catches hallucination without discarding the whole evaluation.

Why Submission and Evaluation are separate

They have different lifecycles — a submission exists the moment it's stored, but an evaluation takes time and can fail. Coupling them into one object would mean either blocking the submission response until evaluation completes (bad UX) or storing partial evaluation state inside the submission (muddies responsibility). Keeping them separate means the submission is never lost, evaluation can be retried independently, and the state machine stays clean.

4. Evaluation Approach

Deterministic Checks (cheap, instant, no AI)

- Submission is non-empty and meets a minimum length threshold.
- Contains at least a couple of class or interface-like tokens (e.g. "class", "interface", "extends", "implements", arrows or colons suggesting relationships). This filters out free-form essays that aren't design attempts.
- Duplicate detection via SHA-256 content hash. If the same content is submitted again, the previous evaluation is returned instead of re-running the AI. This is the idempotency mechanism.
- Submission state transitions: Submitted to Evaluating to Completed or Failed. These are deterministic — no AI involved.

AI Evaluation (judgement-heavy, where LLM adds value)

- The AI evaluator receives the problem requirements, the submission content, and a fixed rubric. The system prompt forces structured JSON output — one object per rubric dimension with criterion, score, evidence, concern, suggestion, and confidence.
- Rubric dimensions: requirement understanding, class responsibilities, coupling/cohesion, encapsulation and interfaces, appropriate use of abstractions/patterns, extensibility, edge cases and testability, quality of explanation.
- The prompt explicitly avoids unconstrained questions like "is this a good design?" — instead it asks the model to score each dimension, quote evidence from the submission, and provide a specific suggestion. This is the structured-prompt-plus-fixed-rubric approach the helping guide recommends.
- After the AI returns results, a deterministic evidence-verification step checks that quoted evidence actually appears in the submission text. Mismatches are flagged but don't discard the evaluation.

Why this split matters

The deterministic checks cost nothing — no API call, no latency, no inconsistency. Running them first means we never waste an AI call on an empty submission or a duplicate. The AI is reserved for what it's actually good at: judging design quality and suggesting improvements. If the AI fails, the deterministic checks still ran, the submission is stored, and the learner sees a clear Failed status with a retry option.

5. Tech Stack & Key Decisions

- Backend: Node.js + Express, MongoDB + Mongoose. The `RubricResult` array inside `Evaluation` is naturally document-shaped — Postgres would need JSONB or joins for no real gain. MongoDB fits the evolving rubric shape without migrations.
- Frontend: React + Vite + Tailwind. Plain `useState` with polling for evaluation status — no React Query needed at this scope.
- AI: Anthropic Claude API. Structured JSON output enforced in the system prompt. A mock evaluator fallback exists for development without API credits.
- Testing: Jest + Supertest. `mongodb-memory-server` for integration tests against a real in-memory Mongo, so state machine and idempotency tests are meaningful.
- Infra: Docker + docker-compose (one app service, one Mongo service). Capped at ~15 minutes of effort.

6. Change Tests

Change Test A: Adding a diagram submission format later

Today the learner submits text. Later, the platform supports a class diagram. How much of the domain model changes? Almost nothing. The `Submission` entity holds a `content` field and a `format` field. Today `format` is "text" and `content` is a string. Adding diagrams means `format` can be "diagram" and `content` can be a serialised representation (JSON, PlantUML, image URL). The `Attempt`, `Problem`, and `Evaluation` entities don't change — they operate on the submission, not its format. The AI evaluator's prompt would handle the new format, but the rubric dimensions and `RubricResult` structure stay the same. The practice flow is completely unaffected.

The key design decision that enables this: **Submission is format-agnostic**. It stores content and a format label. The evaluator is the only component that needs to understand the format. This is a direct result of keeping the domain model focused on the practice loop rather than on the submission medium.

Change Test B: Adding a second evaluator later

Today feedback comes from one AI evaluator. Later, we add a rule-based evaluator or human review. Can we add it without rewriting the practice flow? Yes. The Evaluation entity holds a list of RubricResults and an evaluatorType field. Adding a rule-based evaluator means adding a new type that produces RubricResults in the same shape — same criterion, score, evidence, concern, suggestion, confidence. The Submission, Attempt, and Problem entities don't change. The practice flow doesn't change. The frontend renders RubricResults regardless of who produced them.

The key design decision that enables this: **the RubricResult structure is the contract between evaluator and the rest of the system**. Any evaluator — AI, rule-based, human — that produces results in this shape can be plugged in. The evaluation pipeline (deterministic checks, then evaluator, then evidence verification) treats the evaluator as a swappable component.

7. Handling Slow or Failed Evaluation

- The submission is stored before evaluation starts. If the evaluator crashes, the submission survives. The learner doesn't lose their work.
- The state machine (Submitted, Evaluating, Completed, Failed) gives the frontend a clear status to poll. The learner sees "Evaluating..." while waiting, not a frozen page.
- If evaluation fails, the status is set to Failed and the learner can retry. The idempotency key (content hash) ensures a retry of the same content doesn't create a duplicate submission — it picks up where it left off.
- No queue, no Redis, no background worker. Evaluation runs synchronously within the request. The brief says not to turn this into a distributed-systems project, and for a single demo user, a synchronous try/catch with a state machine is the simplest thing that works.

If this scaled: **the evaluator would be the first component to extract into a worker process** with a simple queue (BullMQ + Redis). But this is a design-note discussion, not something to build for a 2-day MVP — knowing when not to build something is as important as knowing how.

8. Key Trade-offs

- Text over diagrams: Text is the smallest format with enough evidence of design quality. Diagrams need a UML editor — significant UI investment for marginal evaluation benefit. Trade-off: we lose explicit structural/relationship evidence. Mitigation: the rubric asks for relationships in text, and the AI can infer structure from descriptions.
- MongoDB over Postgres: The Evaluation document with its embedded RubricResult array is naturally document-shaped. Postgres would need JSONB (losing relational benefits) or joins for every fetch. Trade-off: no relational constraints. Mitigation: the domain is simple enough for application-level validation.
- Synchronous evaluation over async queue: The brief says monolith is fine. A single demo user doesn't need a queue. Trade-off: the request blocks during AI evaluation (10-30 seconds). Mitigation: the frontend polls status, and the submission is already stored so a timeout doesn't lose data.
- Fixed rubric over adaptive scoring: A fixed rubric gives consistent, comparable feedback across attempts — essential for the delta view to be meaningful. Trade-off: can't adapt to unusual approaches. Mitigation: the "quality of explanation" dimension and suggestion field give the AI room to acknowledge non-standard but valid designs.